



Privacy-Preserving Machine Learning on Federated Graph Database Systems

Team members:

160109A - Dehigaspitiya S.L
160576B - Senevirathne D.B
160711L - Wijesiri R.I.M

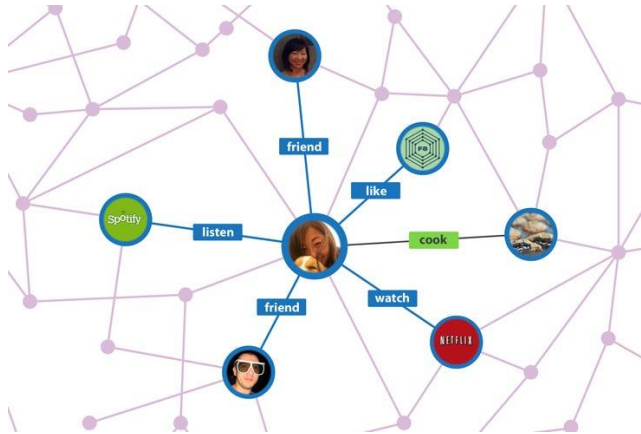
Supervised by:

Dr Miyuru Dayarathna (WSO2)
Prof Sanath Jayasena

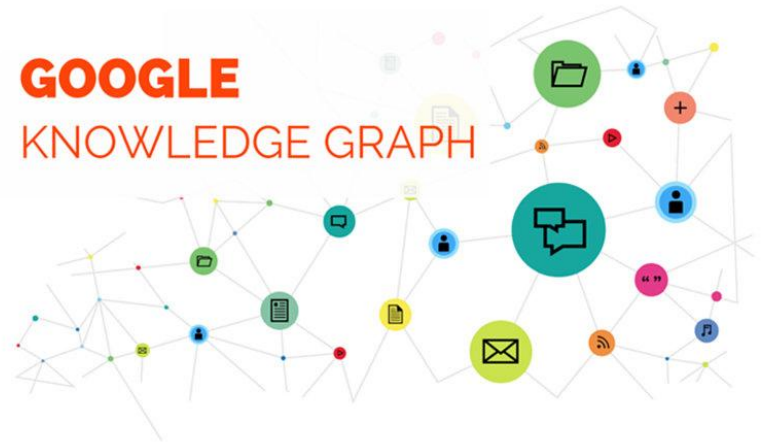
Introduction

Graphs or Networks

Social graphs

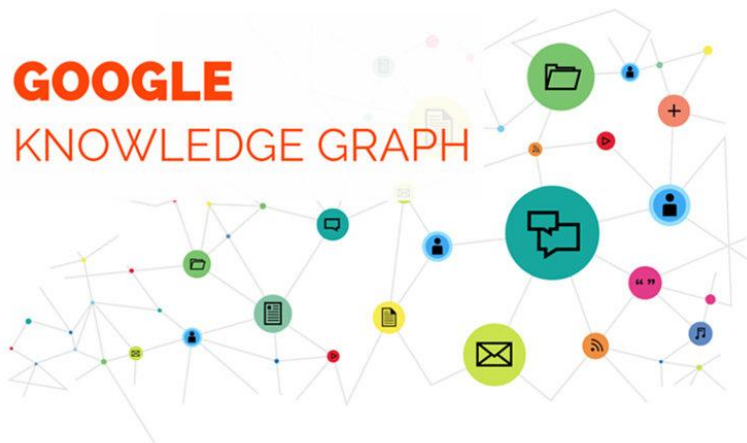


Knowledge graphs



Machine learning on Graphs

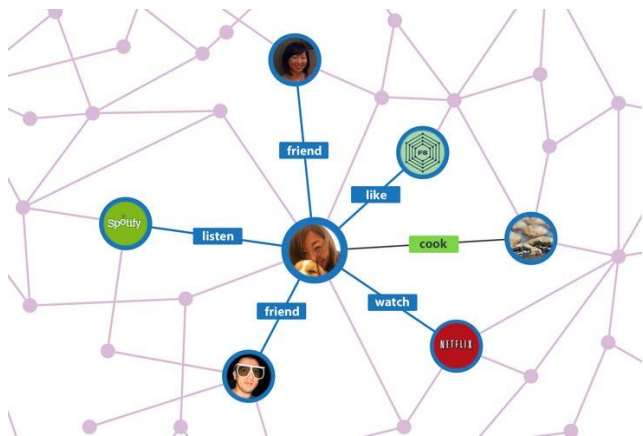
Knowledge graphs



- GCN
- Network embedding a key part of graph-based machine learning
- Learning of embeddings using both features as well as graph structure
- **GCNs** Offer significantly better results in downstream machine learning tasks such as node classification, link prediction, graph clustering etc.

Machine learning on Graphs ctd

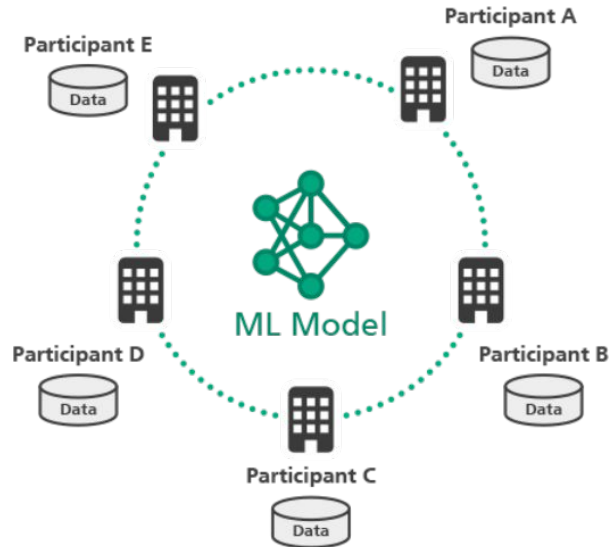
Social graphs



Link Prediction

1. Facebook suggesting new friends
2. Twitter suggesting people to follow
3. LinkedIn suggesting new connections

Collaborative Graph Machine Learning

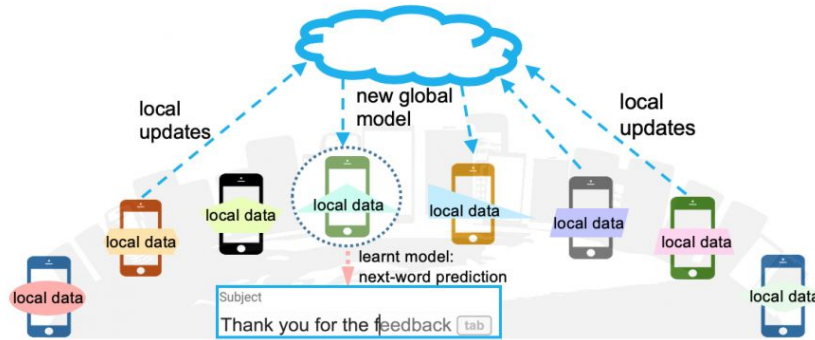


Link Prediction model

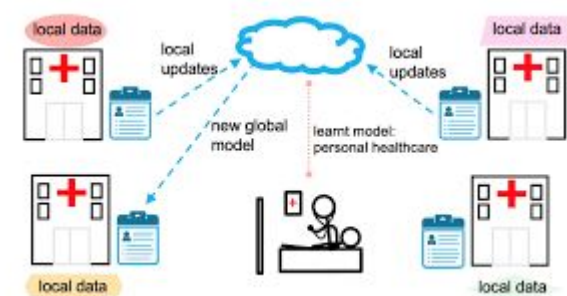
1. Facebook
2. Twitter
3. LinkedIn

Federated Learning

Federated learning among mobile devices



Collaborative learning using Federated learning among Organizations



Research Problem and Solution

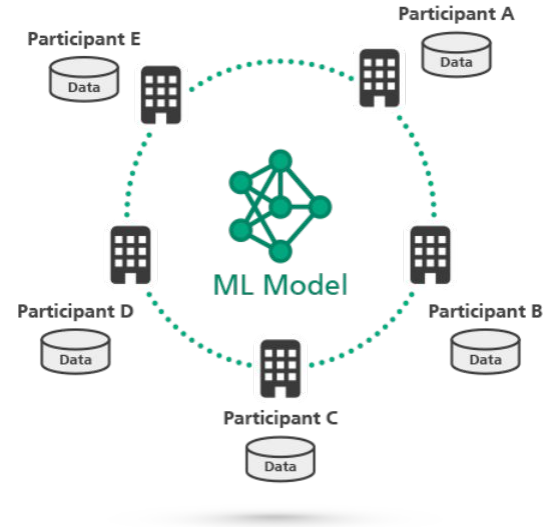


Problem and Context

- Many organizations use Graph data structures to model real world scenarios for many applications (Social media, Search Engines, Medical Organizations)
- Collaborative learning allows organizations to utilize their data for better learning models
- Conducting collaborative, secure and efficient graph learning between organizations is not explored in previous works
- How can we do this?

Proposed Solution

1. A system for conducting collaborative GCN learning between organizations using federated learning
 - Using Federated Learning is not explored.
 - Handling Challenges in graph learning
 - Handling dimensionality mismatch
 - Preserving privacy
 - We propose two approaches for conducting federated learning on graphs
 - **Common Attribute approach**
 - **Common Entity approach**



Proposed Solution Ctd

2. A system to train large scale graphs on commodity hardware

- Easily have billions of nodes and edges
- Extremely resource intensive - Specially Memory
- Using Distributed learning is not explored

Cora	16 mb	2.5 Gb
Twitter	157 mb	3.84 Gb
DBLPV11 citation 2019	9523 mb	107.48 Gb (Estimated)

-> HP9Gen crashed (64 Gb RAM)

Related Work



Related Work

Reference	Contributions	Limitations/Relevance
[2] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” In Conference on Artificial Intelligence and Statistics, 2017.	Introduces federated learning concepts	• Does not focus on the graph learning context • Uses assumptions that are unfit for our problem domain
[5] T. Suzumura et al., “Towards Federated Graph Learning for Collaborative Financial Crimes Detection,” arXiv:1909.12946 [cs, q-fin], Oct.	Fraud detection proposal using graph data	• Is not scalable • Does not use state of the art graph neural networks
[4] "TensorFlow Federated", TensorFlow, 2020. [Online]. Available: https://www.tensorflow.org/federated . [Accessed: 21- Jan- 2020].	Federated environment in Tensorflow High level API for FL simulation	• Only for simulation purposes • No security implemented • Less flexibility



Related Work Ctd

[15] S. Hardy et al., “Private federated learning on vertically partitioned data via entity resolution and additively homomorphic encryption,” arXiv:1711.10677	Introduces vertical federated learning (VFL) and proposes the use of CLK for privacy preserving ER	• Their privacy preserving ER does not focus on the graph learning context • Training of participant data done not locally • Scalability not considered
[11] “FederatedAI/FATE”, GitHub, 2020.	Federated framework Federated AI Technology Enabler (FATE) which supports several ML algorithms to be done in a federated manner	• Graph learning not supported • No support for scalability
[26] T. Ryffel et al., “A generic framework for privacy preserving deep learning,” arXiv:1811.04017	Provides a general framework for federated learning. Emphasis on the privacy aspects.	• Graph learning not supported • No support for scalability



Related Work ctd.

PyTorch-BigGraph (PBG) [16]	Distributed graph training mechanism	• Random node partitioning • Shared file system • GCNs not utilized
Euler [1]	Distributed graph learning framework	• Train models developed in Tensorflow to be trained on heterogeneous graphs • But untested for large graphs like DBLP-V11 • Depends on HDFS based shared file system
JanusGraph [3], Acacia [7] and Trinity [23]	Distributed graph databases	• Distributed processing of graphs • But does not support graph machine learning

[1] Alibaba. 2019. Euler. URL: <https://github.com/alibaba/euler>.

[3] Apache Software Foundation. 2020. JanusGraph. URL: <https://janusgraph.org/>.

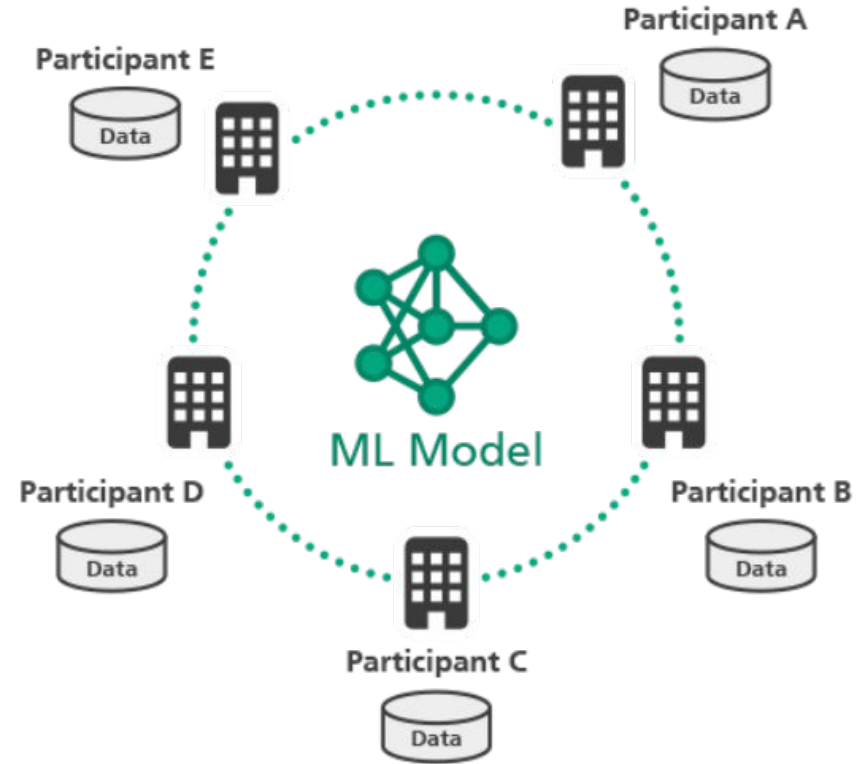
[7] M. Dayarathna and T. Suzumura. 2014. Towards Scalable Distributed Graph Database Engine for Hybrid Clouds. In 2014 5th International Workshop on Data-Intensive Computing in the Clouds. 1–8. <https://doi.org/10.1109/DataCloud.2014.9>

[16] Adam Lerer, Ledell Wu, Jiajun Shen, Timothée Lacroix, Luca Wehrstedt, Abhijit Bose, and Alexander Peysakhovich. 2019. PyTorch-BigGraph: A Large-scale Graph Embedding System. CoRR abs/1903.12287 (2019). arXiv:1903.12287 <http://arxiv.org/abs/1903.12287>

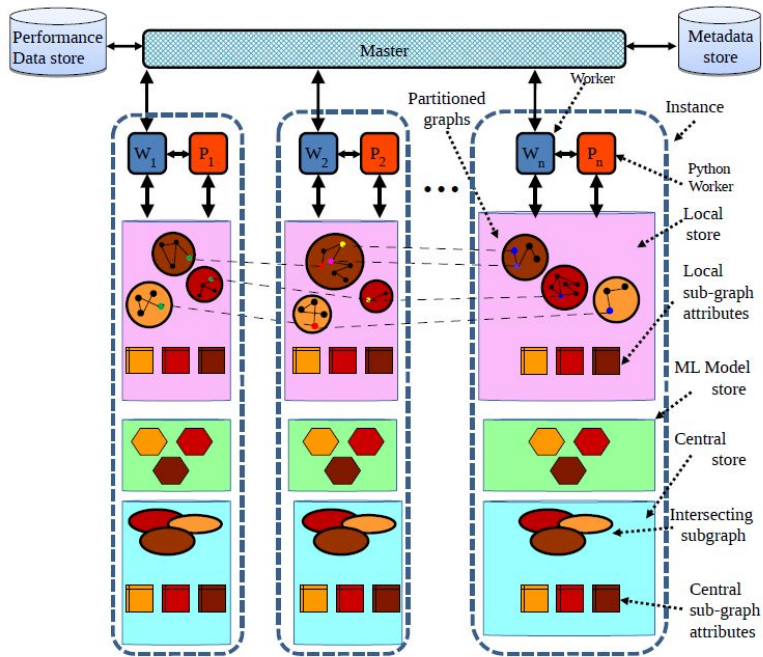
[23] Bin Shao, Haixun Wang, and Yatao Li. 2013. Trinity: A Distributed Graph Engine on a Memory Cloud. In Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD '13). Association for Computing Machinery, New York, NY, USA. 505–516. <https://doi.org/10.1145/2463676.2467799>

Methodology

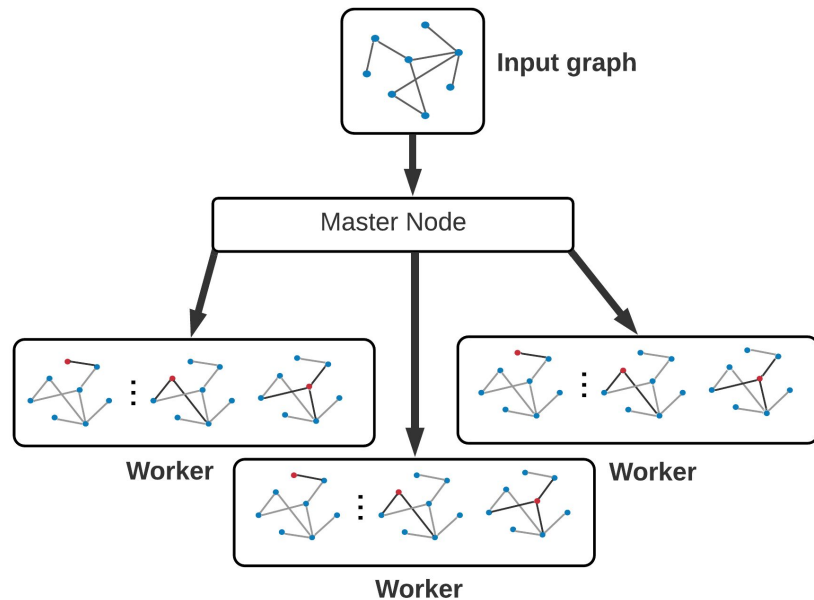
Bigger Picture



Scalable Memory Efficient training within organizations

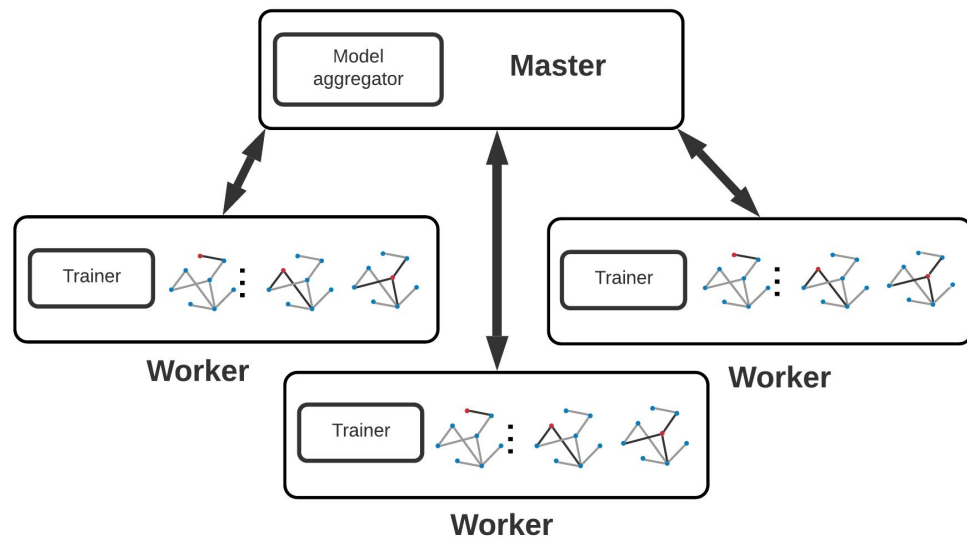


JasmineGraph



Training and Aggregation

1. Initialize Global model and send to workers
2. Workers parallel do training based on scheduling algorithm
3. After training round, send weights to aggregator
4. Repeat



Partition Scheduling

- All graph partitions might not fit into memory at once
- Decides which partitions are to train in parallel in a given moment
- Ensures that memory overflow is avoided
- Packs partitions into memory in a way that the training time is optimized
- Uses best first fit approach

Algorithm 3: Partition Scheduler

```
Input: Map of partition to its memory estimations  $\mathcal{L}$ , List of clients  $C$ , Total memory capacity of host  $c$   
Output: Map of partition to scheduled order per client in host  $S$   
 $sortedMemoryList \leftarrow$  (Sort  $\mathcal{L}$  in descending order)  
 $order \leftarrow 1$   
 $availableMemory \leftarrow c$   
 $freeClients \leftarrow C$   
while  $n(S) < n(sortedMemoryList)$  do  
   $proceedToNextIter \leftarrow 0$   
  foreach  $partition, memory(p, m) \in sortedMemoryList$  do  
    if  $p \notin S$  AND  $m < availableMemory$  AND  $freeClients \neq \emptyset$  then  
       $w \leftarrow$  (Pop client from  $freeClients$ )  
       $S[w][p] \leftarrow order$   
       $availableMemory \leftarrow availableMemory - m$   
       $proceedToNextIter \leftarrow 1$   
       $simulatedMem[w][p] \leftarrow m$  // Client, Partition and memory of partition yet to train  
    end  
  end  
   $order \leftarrow order + proceedToNextIter$   
   $minRemainingMem \leftarrow$  (Minimum value of  $simulatedMem$ )  
  foreach  $client, partition, memory\ yet\ to\ be\ trained (w, p, r) \in simulatedMem$  do  
     $m \leftarrow \mathcal{L}[p]$   
    if  $r = minRemainingMem$  then  
       $availableMemory \leftarrow availableMemory + m$   
      Remove  $(p, r)$  from  $simulatedMem$ ;  
      Insert  $w$  into  $freeClients$ ;  
    else  
       $simulatedMem[w][p] \leftarrow r - minRemainingMem$   
    end  
  end  
end  
return  $S$ 
```

Memory estimation

Algorithm 4 Memory estimation

Input: List of graph partition - $\mathcal{G}$, Size of in-memory data type used in bytes - m_E, m_N, m_F

Output: Map of estimated memory usage of Python clients in GB when training each graph partition - $\mathcal{L}_{\mathcal{M}}$

$\mathcal{L}_{\mathcal{M}} \leftarrow \{\}$;

for each partition $g \in \mathcal{G}$ **do**

$\mathcal{N}_N \leftarrow$ (Number of nodes in g)

$\mathcal{N}_E \leftarrow$ (Number of edges in g)

$\mathcal{N}_F \leftarrow$ (Number of node features in g)

$M_N \leftarrow \mathcal{N}_N * m_N$

$M_E \leftarrow 2 * \mathcal{N}_E * m_E$

$M_F \leftarrow \mathcal{N}_N * \mathcal{N}_F * m_F$

$inMemGraphSize \leftarrow (M_N + M_E + M_F) / (1024 * 1024 * 1024)$;

$\mathcal{L}_{\mathcal{M}}[g] \leftarrow \mathcal{F}(inMemGraphSize)$;

end

- Estimate partition size in memory based on number of nodes, edges and attributes

$(\mathcal{F}(inMemGraphSize) = K_1 * inMemGraphSize + K_2$
with $K_1 = 3.53$ and $K_2 = 1.58$).

Training and Aggregation

Algorithm 2 Federated model aggregation - Server

Input: Model - M , Set of workers/hosts - $\mathcal{H}$, Number of Rounds - $\mathcal{R}$

Output: Trained Global model

$\mathcal{M} \leftarrow \text{INITIALIZE}(M)$;

$w_0 \leftarrow \mathcal{M}.\text{weights}$;

for each round $r = 1, 2, \dots, R$ **do**

$\mathcal{W} \leftarrow []$;

for each worker $h \in \mathcal{H}$ **in parallel do**

for each client $c \in h$ **in parallel do**

$w \leftarrow \text{CLIENTTRAIN}(c, w_r)$;

$\mathcal{W}.\text{extend}(w)$;

end

end

$w_{r+1} \leftarrow \sum_{k=1}^{\mathcal{W}.\text{length}} \frac{n_k}{n} \mathcal{W}[k]$;

end

$\mathcal{M}.\text{weights} \leftarrow w_R$;

Algorithm 1: Federated training - Workers/Hosts

Input: Model - M , List of graph partitions in the host - $\mathcal{G}$, List of clients in the host - $\mathcal{C}$, Total memory capacity of host - h , Number of local epochs - $\mathcal{E}$, Learning rate - η , Sample percentage - $\mathcal{P}$, Batch size - B

Output: Trained Local model for every graph partition

$\mathcal{G} \leftarrow (\text{reconstruct partitioned graphs of } \mathcal{G} \text{ with edge cuts})$

$\mathcal{L} \leftarrow \text{MEMESTIMATE}(\mathcal{G})$; // Memory estimation per

partition using the memory estimation algorithm

$\mathcal{S} \leftarrow \text{SCHEDULE}(\mathcal{L}, \mathcal{C}, h)$; // Using the scheduling algorithm

$\mathcal{M} \leftarrow \{\}$;

for each graph $g \in \mathcal{G}$ **do**

$\mathcal{M}[g] \leftarrow \text{INITIALIZE}(M)$;

end

Function $\text{CLIENTTRAIN}(c, w)$:

$\mathcal{W} \leftarrow []$;

for each graph partition $g \in \mathcal{S}[c]$ **do**

$\mathcal{M}[g].\text{weights} \leftarrow w$;

$graph \leftarrow (\text{Load } g \text{ to memory})$;

$\mathcal{B} \leftarrow (\text{sample the } graph \text{ and split batches with sample percentage } \mathcal{P} \text{ and batch size } B)$;

for each epoch $e = 1, 2, \dots, \mathcal{E}$ **do**

for each batch $b \in \mathcal{B}$ **do**

$\text{TRAIN}(\mathcal{M}, \eta)$; // $w \leftarrow w - \eta \nabla l(w; b)$

end

end

 Insert $\mathcal{M}[g].\text{weights}$ into $\mathcal{W}$;

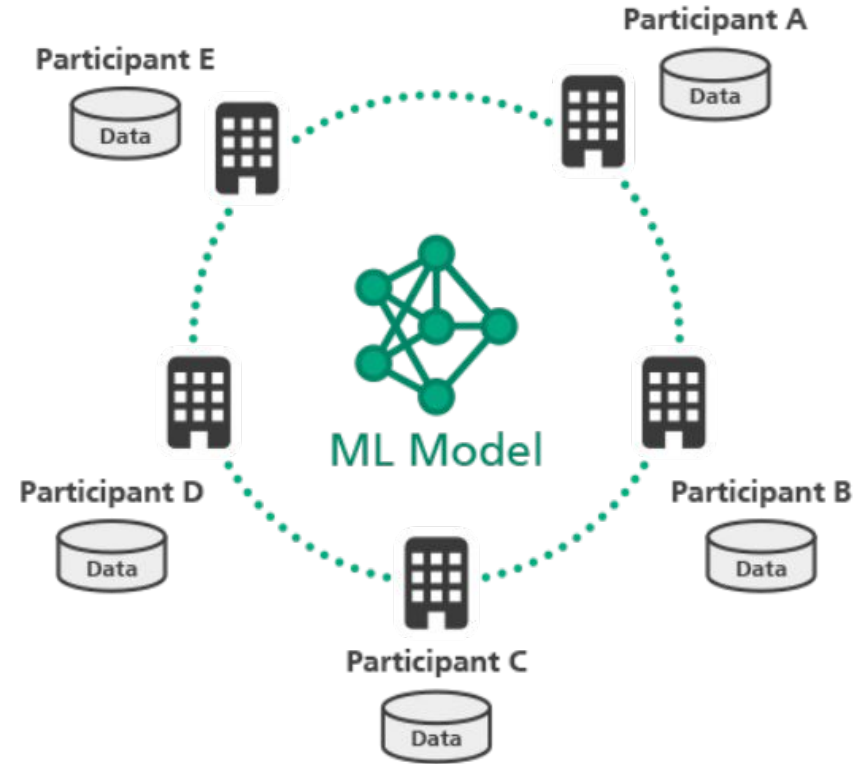
 Delete $graph$ from memory;

end

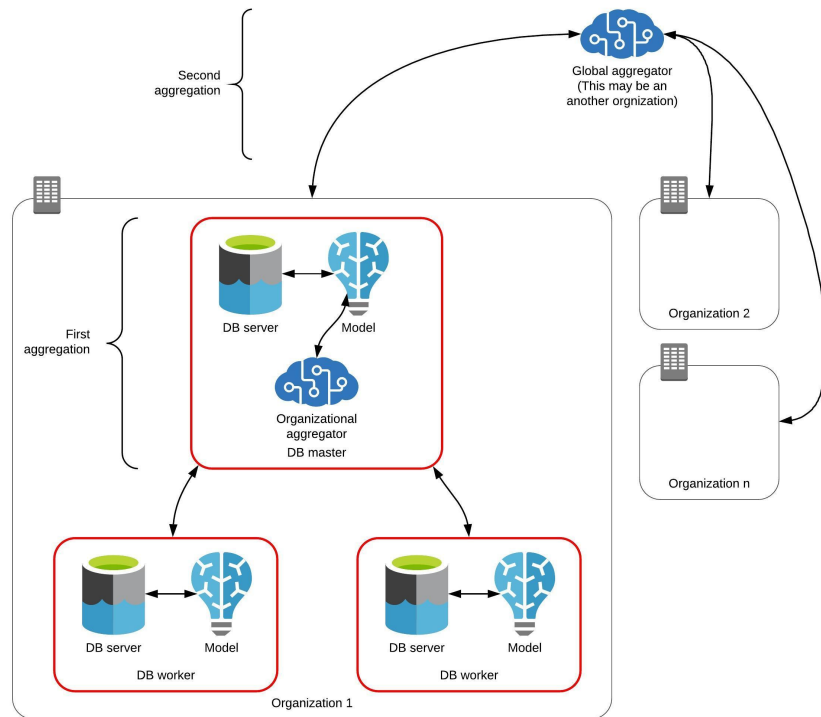
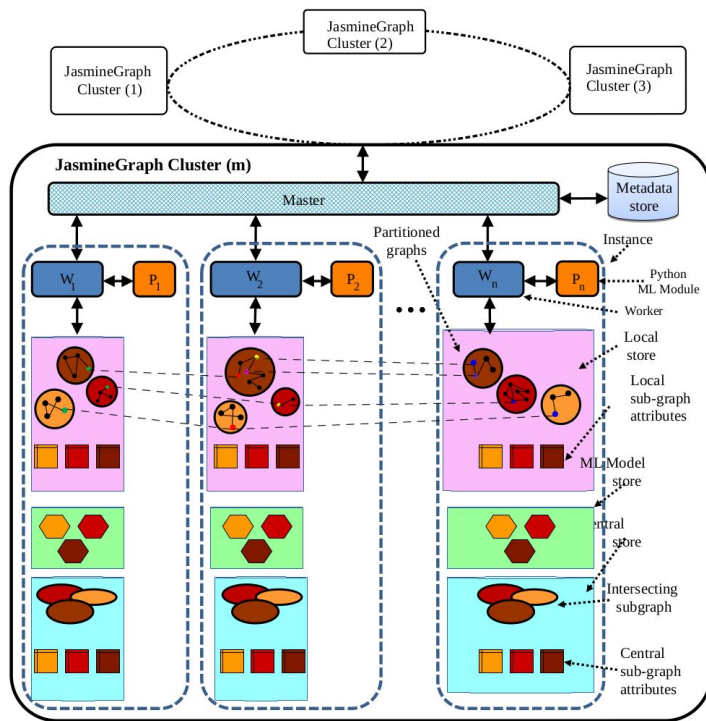
 return $\mathcal{W}$;



Bigger Picture



Collaborative Graph Learning among organizations



Collaborative Graph Learning : Aggregation

Algorithm 6 Global federated model aggregation - Global aggregator

Input: Model - M , Set of organizations - $\mathcal{O}$, Number of global rounds - $\mathcal{R}$

Output: Trained Global model

$\mathcal{M} \leftarrow \text{INITIALIZE}(M);$

$w_0 \leftarrow \mathcal{M}.\text{weights};$

for each round $r = 1, 2, \dots, R$ **do**

$\mathcal{W} \leftarrow [];$

for each organization $o \in \mathcal{O}$ **in parallel do**

$w \leftarrow \text{ORGANIZATIONTRAIN}(c, w_r);$

$\mathcal{W}.\text{extend}(w);$

end

$w_{r+1} \leftarrow \sum_{k=1}^{\mathcal{W}.\text{length}} \frac{n_k}{n} \mathcal{W}[k];$

end

$\mathcal{M}.\text{weights} \leftarrow w_R;$

Algorithm 5 Federated model aggregation - Organizational server

Input: Model - M , Set of workers/hosts - $\mathcal{H}$, Number of local rounds - $\mathcal{R}$

Output: Trained Organizational model

Function ORGANIZATIONTRAIN(c, w_g):

$w_0 \leftarrow w_g;$

for each round $r = 1, 2, \dots, R$ **do**

$\mathcal{W} \leftarrow [];$

for each worker $h \in \mathcal{H}$ **in parallel do**

for each client $c \in h$ **in parallel do**

$w \leftarrow \text{CLIENTTRAIN}(c, w_r);$

$\mathcal{W}.\text{extend}(w);$

end

end

$w_{r+1} \leftarrow \sum_{k=1}^{\mathcal{W}.\text{length}} \frac{n_k}{n} \mathcal{W}[k];$

end

 return $w_R;$



Challenges of Graph collaborative learning

Dimentionality Mismatch of graph datasets among Organizations

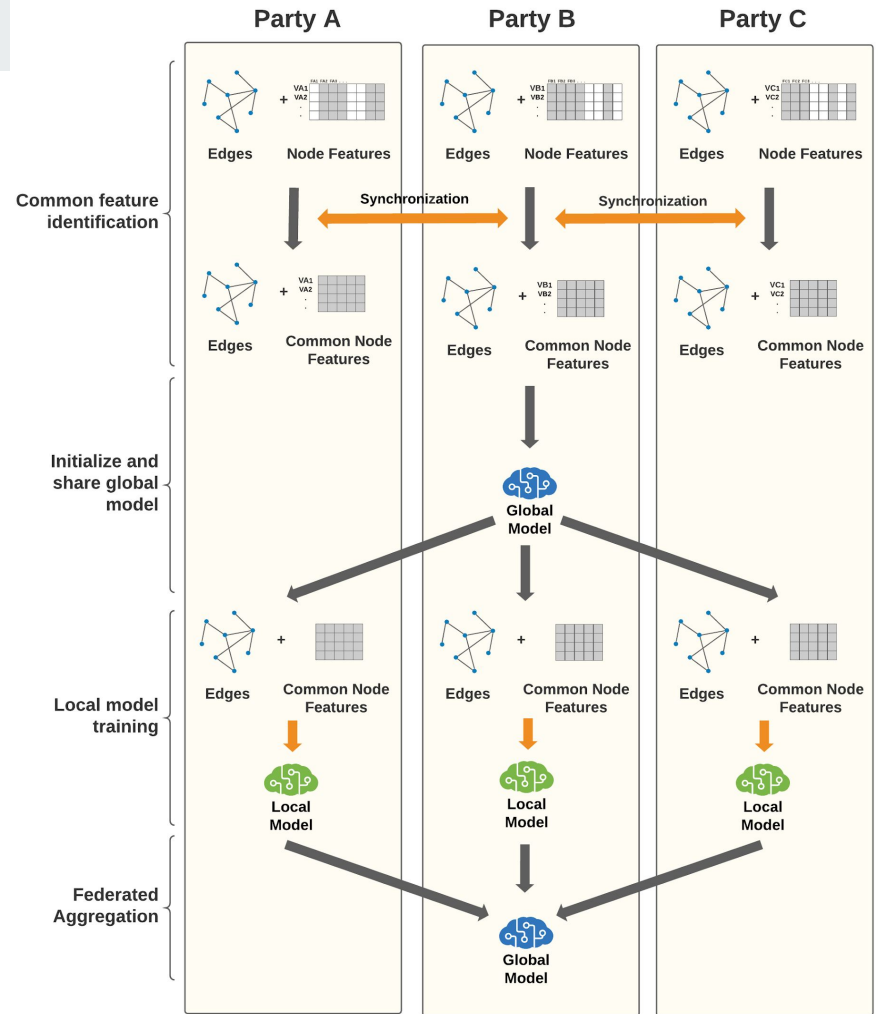
1. Common Attribute approach
2. Common Entity approach

Privacy of data is in risk

1. Common Entity approach

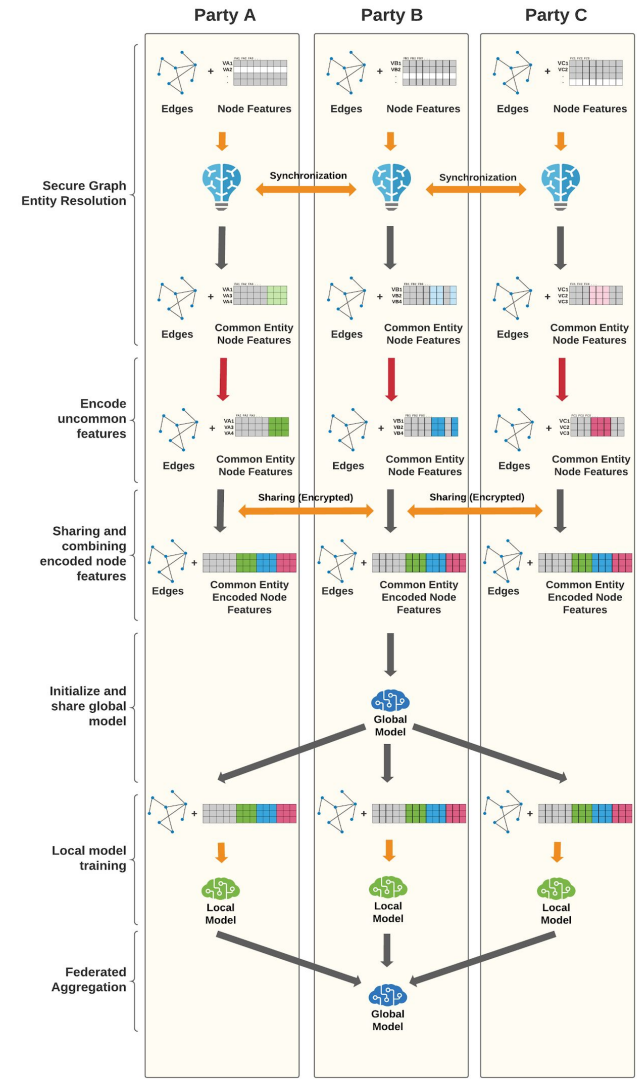
Common Attribute Approach

- Organizations agree beforehand what are the node features to select
- Select common node features of organizations as preprocessing step
- Use selected node features for training



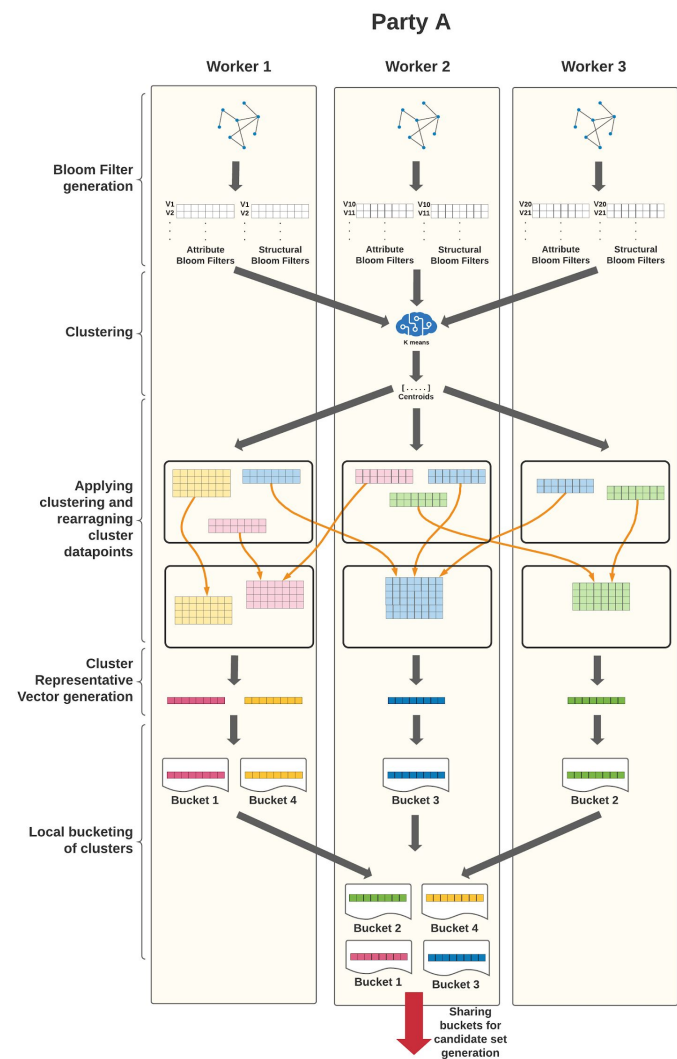
Common Entity Approach

- Organizations securely determine the common entities (vertices)
- Select common entities as preprocessing step
- Encode features of common entities to share and combine
- Use new node features of common entities as training data



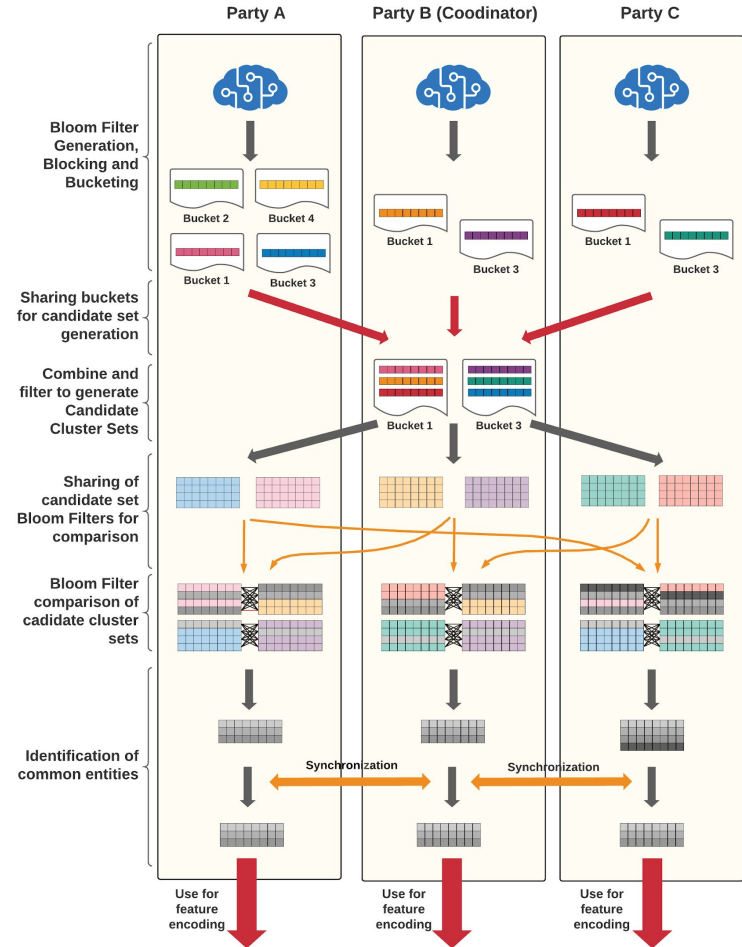
Secure Graph Entity Resolution

- Idea is to securely and efficiently compare entities
 - Create Bloom Filters for entities
 - Cluster/Block Bloom Filters
 - Compare Bloom Filters of similar clusters
- Distributed among workers
- Organizations process independently until cluster sharing step



Secure Graph Entity Resolution ctd.

- Use a circular sharing strategy to compute intermediate common entities
 - Drastically reduces communication needed and computation required
- Use similar strategy to combine intermediate results to get common entities across all parties



Secure Graph Entity Resolution ctd.

Algorithm 5: Comparison Protocol of single party

Input: Local cluster of Bloom Filters in the candidate set C ,
 Cluster of Bloom Filters in the candidate set
 belonging to the previous party in the chain C_{prev} ,
 Similarity threshold for Dice coefficient d , Next
 party in the chain P_{next}

Output: Intermediate common entity map of Local entity
 cluster id to cluster ids from previous party in the
 chain $\mathcal{H}$

SHARE(C, P_{next})

$\mathcal{H} \leftarrow \{\}$

foreach Bloom filter $f_1 \in C$ **do**

$T \leftarrow []$

foreach Bloom filter $f_2 \in C_{prev}$ **do**

$c \leftarrow f_1 \cdot f_2$ // Count of common '1' positions

$n \leftarrow \text{sum}(f_1 + f_2)$ // Count of '1' marked positions
 of both

$s \leftarrow 2 * c / n$ //Dice coefficient between f_1 and f_2

 (Insert s to T)

end

$i \leftarrow \arg \max_T$

if $T[i] \geq d$ **then**

$\mathcal{H}[\arg f_1] \leftarrow i$

end

end

return $\mathcal{H}$

Algorithm 6: Synchronization

Input: List of combined intermediate pairwise common
 entity maps given by each organization $\mathcal{L}$, Circular
 linked list of parties $\mathcal{P}$, Coordinator party p_{corr}

Output: Map of party to its common entities $\mathcal{H}$

$p_{current} \leftarrow \mathcal{P}.head()$

$p_{next} \leftarrow \mathcal{P}.head().next()$

$Q_{current} \leftarrow \mathcal{L}[p_{current}].keys()$

//First pass

while $p_{next} \neq \mathcal{P}.head()$ **do**

$Q_{next} \leftarrow []$

foreach Entity $q \in Q_{current}$ **do**

if $q \in \mathcal{L}[p_{current}]$ **then**

 (Insert $\mathcal{L}[p_{current}][q]$ to Q_{next})

end

end

$Q_{current} \leftarrow Q_{next}$

$p_{current} \leftarrow p_{next}$

$p_{next} \leftarrow p_{next}.next()$

end

$p_{current} \leftarrow \mathcal{P}.head()$

$p_{next} \leftarrow \mathcal{P}.head().next()$

$\mathcal{H} \leftarrow \{\}$

// Second pass

while $p_{next} \neq \mathcal{P}.head()$ **do**

$\mathcal{H}[p_{current}] \leftarrow Q_{current}$

$Q_{next} \leftarrow []$

foreach Entity $q \in Q_{current}$ **do**

 (Insert $\mathcal{L}[p_{current}][q]$ to Q_{next})

end

$Q_{current} \leftarrow Q_{next}$

$p_{current} \leftarrow p_{next}$

$p_{next} \leftarrow p_{next}.next()$

end

return $\mathcal{H}$

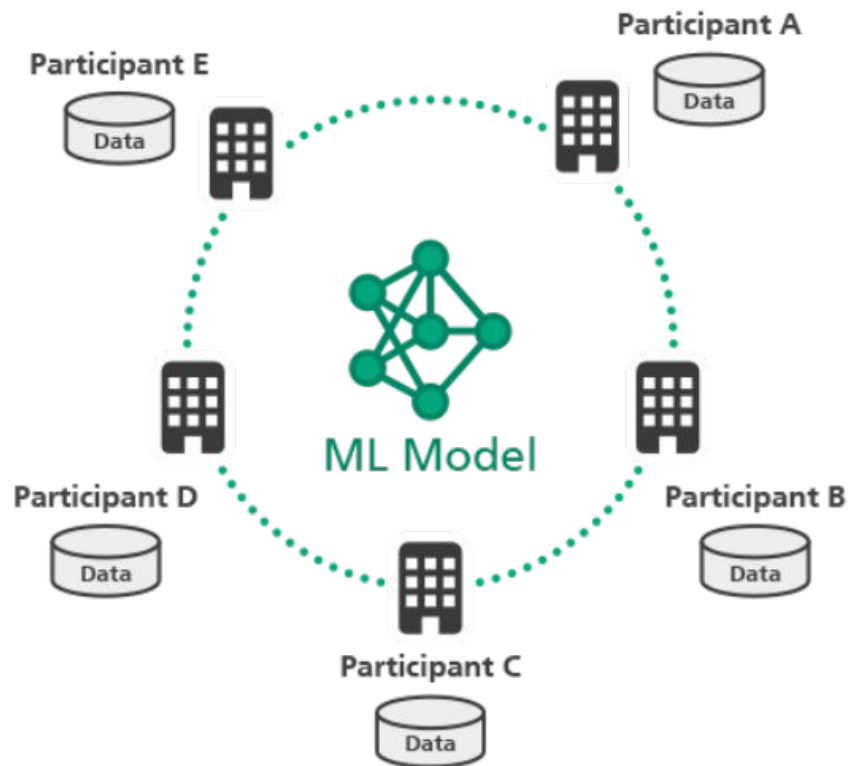


Common Attribute approach vs Common Entity approach

Common Attribute approach	Common Entity approach
Is able to utilize (securely) all of the data points scattered across organizations	Is only able to utilize the data points scattered across organizations that correspond to the common entities
Is unable to utilize all of the useful and unique node feature information coming from all organizations	Can utilize all of the unique node feature information that is given by organizations (for common entities only)

Evaluation

Evaluation Bigger Picture





Single organization training

- General Model Performance Evaluation.
- Partitioned Training Evaluation
 - Assign one partition per client
 - Assign multiple partitions per client
- Implementation on Large scale graphs



Datasets - Single organization training

Data Set	Vertices	Edges	No of features	Edge file size(MB)	Feature file size(MB)	Space in memory (GB)
DBLP-V11 ^a	4,107,340	36,624,464	948	508	9523	107.5 (Estimate)
Reddit ^b	232,965	11,606,919	602	145	270	3.84
Twitter ^c	81,306	1,768,149	1007	16	157	2.5

- Original sources:
 - a. <https://www.aminer.org/citation>
 - b. <http://snap.stanford.edu/graphsage/>
 - c. <https://snap.stanford.edu/data/ego-Twitter.html>
- Our prepared versions available at <https://github.com/limetreestate/graph-datasets>



Datasets - Single organization training ctd.

Link prediction predicts whether there will be links between two nodes based on the attribute information and the observed existing link information.

Twitter: Suggest new users to follow

- Nodes - Twitter Users
- Edges (Directed) - User follows another
- Features - Twitter handles and hashtags used in user node's tweets

Reddit: Recommend content/posts that user might find interesting

- Nodes - Reddit posts
- Edges - There are common users between two posts
- Features - Extracted from textual content of the post node



Datasets - Single organization training ctd.

DBLP-V11: Suggest new papers that a researcher might find useful/interesting

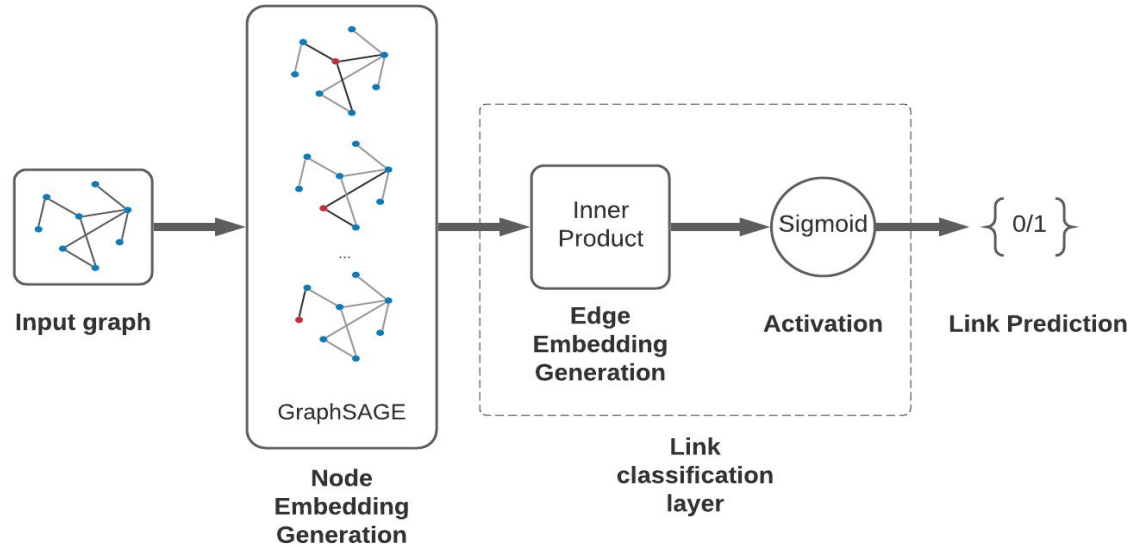
- Nodes - Research papers
- Edges (Directed) - One paper cites the other in its work
- Features - The field(s) of study that the paper node belongs to



Experiment Environment

Processor	Intel®Xeon®CPU E7-4820 v3 @ 1.90GHz, 40 CPU cores (80 hardware threads via hyperthreading),
Main memory	64GB RAM
Cache memory	32KB L1 (d/i) cache, 256K L2 cache, and 25600K L3 cache.
Storage	1.8TB hard disk drive
Operating System	Ubuntu Linux version 16.04 with Linux kernel 4.4.0-148-generic.

Model



- Generate node embeddings for nodes of a potential links
- Generate link/edge representation using inner product
- Classify potential link



General Model Performance

The following numbers reflect how a unpartitioned and trained link prediction model performs on these datasets

Dataset	Accuracy	Recall	AUC	F1	Precision
Twitter	0.7887	0.9869	0.9576	0.8350	0.7233
Reddit	0.7174	0.9026	0.8037	0.7616	0.6587
DBLP-V11	Cannot train in conventional setting, crashes				

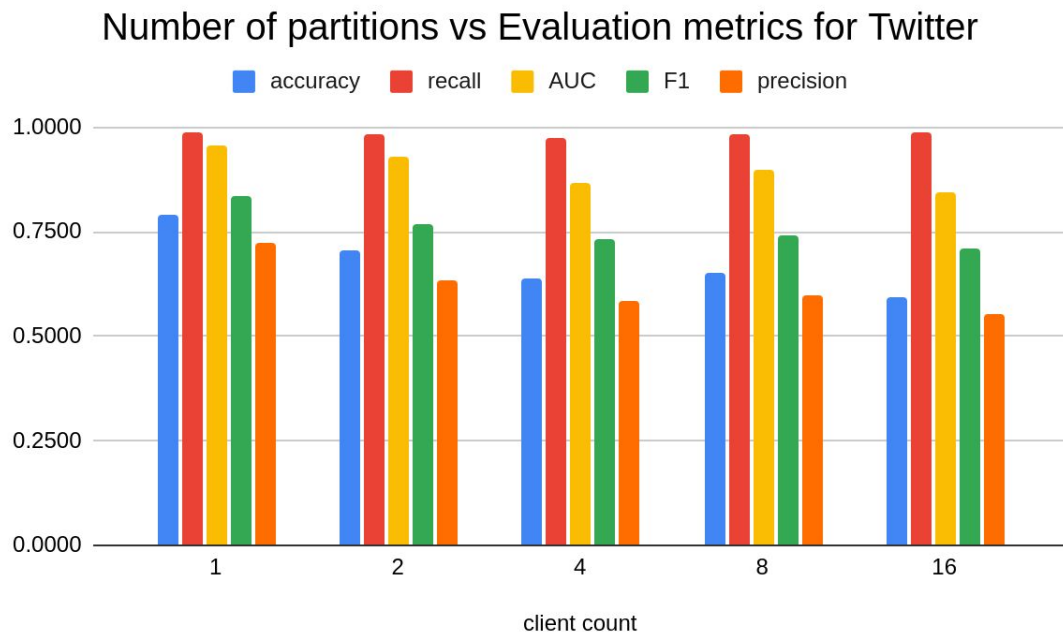


Model performance (Twitter)

For following table, used client count is equal to the number of partitions

Partition count	Accuracy	Recall	AUC	F1 Score	Precision
1 (unpartitioned)	0.7887	0.9869	0.9576	0.835	0.7233
2	0.7047	0.9831	0.9292	0.77	0.6336
4	0.6395	0.973	0.8672	0.7306	0.5861
8	0.6537	0.9844	0.8977	0.7412	0.5962
16	0.5936	0.986	0.8441	0.7088	0.5538

Model performance (Twitter) ctd.



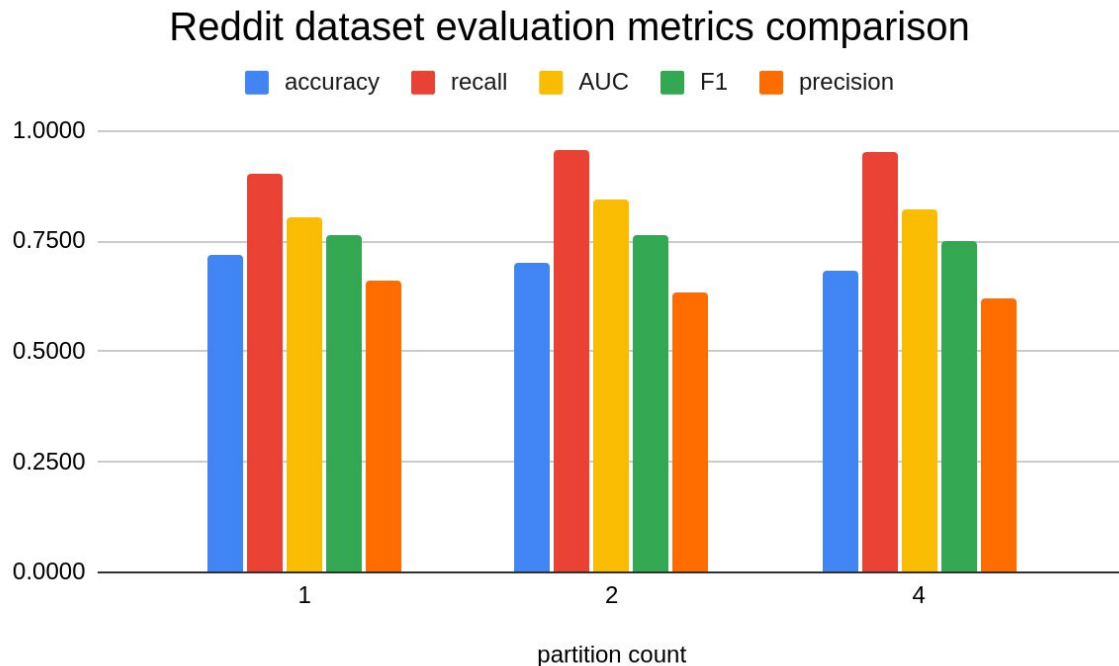


Model performance (Reddit)

Client count is equal to the number of partitions in the following results

Partition count	Accuracy	Recall	AUC	F1 Score	Precision
1 (unpartitioned)	0.7174	0.9026	0.8037	0.7616	0.6587
2	0.702	0.9559	0.8458	0.7625	0.6344
4	0.6836	0.9534	0.8201	0.751	0.6202

Model performance (Reddit) ctd.



Elapsed Training Times

Following table contains results related to twitter/Reddit dataset using 16 partitions. Trained for 5 rounds with 3 epochs per round.

Number of clients	Elapsed Time (seconds)
1 (unpartitioned graph)	10h:31m:48s
2	05h:26m:15s
4	03h:35m:22s

Twitter dataset



Time reduced : 6 hours, 56 minutes (65.91%)

Number of clients	Elapsed Time (seconds)
1 (unpartitioned graph)	09h:08m:03s
2	06h:06m:51s
4	04h:10m:19s

Reddit dataset



Time reduced : 4 hours, 57 minutes (54.325%)



Implementation on Large Graphs (DBLP-V11)

- We were unable to train DBLP-V11 using conventional training
- But using proposed solution (with scheduling) we were able to train DBLP-V11 using 16 partitions with 2 clients (20.5 hours)
- But due to a memory growth in the system we trained DBLP-V11 dataset by using two steps (3 training rounds and then 2 i.e 15 epochs total)

Dataset	Accuracy	Recall	AUC	F1	Precision
DBLP-V11	0.56529	0.99584	0.88943	0.69677	0.53630



Collaborative Training - Federated Datasets

- Synthesized several compatible simulated organizational datasets using a real world dataset¹
- Created several scenarios, with 3 organizations per scenario
- Varied the number of common entities and common attributes between scenarios
- Idea was to vary the common entities from LOW (~20% of source dataset) to MID (~50%) to HIGH (~80%). Same with common attributes

1. Our prepared federated datasets are available at [federated-graph-datasets/twitter at master · limetreestate/federated-graph-datasets](https://limetreestate.com/federated-graph-datasets/)



Collaborative Training - Federated Datasets ctd.

Common entity scenarios	Edges Per Org. (approx)	Edges for common entities per Org. (approx)	Nodes per Org. (approx)	Common nodes between organizations
LOW	800,000	410,000	45,000	27,460
MID	930,000	700,000	54,000	40,655
HIGH	1,110,000	1,050,000	70,000	65,086

Common attributes scenarios	Per Organization	Common
LOW	700	301
MID	993	740
HIGH	1,321	1,232

NOTE: Source/Combined dataset (Twitter) has 81306 vertices, 1768149 edges and 1500 features per node



Collaborative Training Between Multiple Organizations

Number of global aggregation steps during training process	3
Number of training rounds per organization	4
Number of epochs per training rounds	3

- GCN model with the same hyperparameters(GraphSAGE) and sampling rates(training sampling rate 0.02 and testing sampling rate 0.01) were used for the following steps.



Scenario 01 - average F1 comparison (Low common attributes & Low common entity)

Organization id	Single organization training	Common attributes training		Common entity training	
Organization 01	0.7785	0.7862	0.0077+	0.7626	0.0159-
Organization 02	0.7656	0.7784	0.0128+	0.7744	0.0088+
Organization 03	0.7191	0.7383	0.0192+	0.7685	0.0494+



Scenario 02 - average F1 comparison (Medium common attributes & Medium common entity)

Organization id	Single organization training	Common attributes training		Common entity training	
Organization 01	0.7675	0.8049	0.0374+	0.7828	0.0153+
Organization 02	0.7589	0.7983	0.0394+	0.7747	0.0158+
Organization 03	0.7401	0.7678	0.0277+	0.7697	0.0296+



Scenario 03 - average F1 comparison (High common attributes & High common entity)

Organization id	Single organization training	Common attributes training		Common entity training	
Organization 01	0.7681	0.8101	0.0420+	0.8119	0.0438+
Organization 02	0.7649	0.8042	0.0393+	0.8088	0.0439+
Organization 03	0.7628	0.8005	0.0377+	0.8064	0.0436+

Discussion



Collaborative Learning

- Both approaches achieve better results compared to standalone training
- Common Attribute approach performs better for LOW and MID common entity scenarios compared to the Common Entity approach
 - Overfitting¹
 - Less datapoints for Common Entity approach
- Common Entity approach performs better when there are enough data points in the training data to generalize well to the entire data.

1. [Do we need deep graph neural networks? | by Michael Bronstein | Towards Data Science](#)

Conclusion



Conclusion

- Can any graph machine learning model for any task
 - We evaluate on using an offline developed model for link prediction
- Reduced time taken for training by partitioning and scheduling
 - DBLP-v11 dataset (>10GB) trained for 15 epochs in **20 hours 24 minutes** with 16 partitions and 2 workers where conventional training couldn't process it at all
 - Reddit trained in 4 hours 10 minutes (16 partitions, 4 workers). Conventional took 9 hours 8 minutes



Conclusion ctd.

- We propose two approaches for federated learning between graphs
 - Common Attribute approach
 - Common Entity approach
- Both perform better than local organization
- Common Attribute approach perform better with Low and Mid common entities
- Common Entity approach performs better with high entity approach

Publications



Research publications

- Research paper "Memory Efficient Graph Convolutional Network based Distributed Link Prediction" accepted at 2020 BigGraphs Workshop at IEEE BigData'20

Thank You!